



GUIA INTRODUTÓRIO - EDIÇÃO 2026

Fundamentos para a Certificação Data Engineer Associate

Databricks - roteiro de estudo para iniciantes

ORIENTAÇÃO

Antes de começar

Este guia organiza os fundamentos cobrados na certificação Databricks Certified Data Engineer Associate. Ele foi escrito para quem está iniciando e precisa transformar o guia oficial em uma rota de estudo.

O material não substitui prática na plataforma, treinamento oficial nem o guia vigente do exame. Produtos, nomes e objetivos podem mudar; confira novamente a página oficial antes de agendar a prova.

Ao final, você deverá reconhecer os principais componentes da plataforma, entender o percurso de um dado em uma arquitetura lakehouse e saber quais atividades praticar em cada domínio.

Fonte: Databricks Certified Data Engineer Associate Exam Guide, versão válida a partir de 4 maio 2026.

MAPA DE ESTUDO

Como o exame está organizado

O guia oficial vigente informa 45 questões pontuadas, 90 minutos, formato de múltipla escolha e ausência de pré-requisito obrigatório. A Databricks recomenda experiência prática antes da tentativa.

Domínio de estudo	O que revisar
Plataforma	workspace, arquitetura e capacidades
Ingestão e carga	fontes, formatos, batch e streaming
Transformação e modelagem	SQL, PySpark, Delta e arquitetura medalhão
Produção	Lakeflow Jobs, pipelines e CI/CD
Operação	monitoramento, solução de problemas e otimização
Governança e segurança	Unity Catalog, permissões e qualidade

Fonte: Guia oficial do exame, maio de 2026.

1 · PLATAFORMA

A plataforma em uma visão única

A Databricks Data Intelligence Platform reúne engenharia de dados, análise, machine learning e inteligência artificial sobre uma base lakehouse. Para a prova, pense menos em uma lista de produtos e mais no fluxo completo do dado.

Fontes ingestão armazenamento transformação consumo governança

- O workspace organiza notebooks, consultas, jobs e outros ativos de trabalho.
- A computação executa consultas e transformações sem precisar estar acoplada ao armazenamento.
- Delta Lake fornece tabelas confiáveis sobre armazenamento de objetos.
- Unity Catalog centraliza descoberta, permissões, linhagem e governança de dados e IA.

Fonte: Databricks documentation: What is Databricks?; What is a data lakehouse?

2 · LAKEHOUSE

Lakehouse combina flexibilidade e controle

Um data lake aceita grande variedade de dados e escala com baixo acoplamento. Um data warehouse favorece dados estruturados, governança e consumo analítico. O lakehouse busca reunir essas propriedades em uma única arquitetura.

No Databricks, o Apache Spark oferece processamento distribuído, enquanto Delta Lake adiciona uma camada de armazenamento otimizada com transações ACID e controle de esquema.

Para estudar, diferencie responsabilidades: Spark processa; Delta Lake estrutura e protege tabelas; Unity Catalog governa ativos.

Fonte: Databricks documentation: What is a data lakehouse?; What is Delta Lake?

3 · MODELAGEM

Bronze, prata e ouro indicam qualidade

A arquitetura medalhão organiza o dado em camadas que aumentam progressivamente sua qualidade e utilidade.

Camada	Estado	Uso típico
Bronze	bruto e rastreável	preservar a chegada e permitir reprocessamento
Prata	limpo e validado	padronizar tipos, remover duplicidades e integrar fontes
Ouro	agregado e orientado ao negócio	alimentar indicadores, relatórios e produtos de dados

A camada não é definida pelo nome do arquivo, e sim pelo nível de refinamento e pelo contrato que oferece aos consumidores.

Fonte: Databricks documentation: [What is the medallion lakehouse architecture?](#)

4 · INGESTÃO

Ingestão começa pela natureza da fonte

Antes de escolher uma ferramenta, responda: os dados chegam em lote ou continuamente? O esquema é estável? É necessário reprocessar? Qual latência o consumidor exige?

- Batch: cargas periódicas e conjuntos delimitados.
- Streaming: eventos contínuos tratados de forma incremental.
- Auto Loader: ingestão incremental de novos arquivos em armazenamento de nuvem.
- COPY INTO: carga idempotente de arquivos para tabelas Delta em cenários adequados.

Na prática, carregue um CSV pequeno, observe o esquema inferido e depois declare explicitamente os tipos esperados. Compare o resultado quando chega uma coluna inesperada.

Fonte: Databricks documentation: ETL quick start; Auto Loader; COPY INTO.

5 · TRANSFORMAÇÃO

Transformações devem ser legíveis e testáveis

A prova exige reconhecer transformações com SQL e PySpark. Concentre-se em seleção, filtros, junções, agregações, tratamento de nulos e criação de colunas.

```
from pyspark.sql import functions as F

resultado = (vendas
    .filter(F.col('valor') > 0)
    .groupBy('produto')
    .agg(F.sum('valor').alias('receita')))
```

Entenda o resultado de cada etapa antes de memorizar métodos. Em SQL, pratique a mesma lógica com WHERE, GROUP BY, funções de agregação e junções.

Prefira funções nativas de Spark a UDFs quando houver equivalente: o otimizador consegue compreender e melhorar o plano de execução.

Fonte: Databricks documentation: PySpark transformations; Spark SQL.

6 · DELTA LAKE

Delta Lake torna pipelines confiáveis

Tabelas Delta mantêm um log de transações associado aos arquivos de dados. Isso possibilita garantias ACID, evolução e validação de esquema e operações de atualização.

- MERGE combina inserções e atualizações de maneira declarativa.
- Time travel permite consultar versões anteriores dentro da retenção disponível.
- Schema enforcement bloqueia gravações incompatíveis.
- OPTIMIZE reorganiza arquivos para melhorar determinados padrões de leitura.

Não confunda confiabilidade lógica com desempenho automático: formato, tamanho de arquivos, filtros e padrão de consulta continuam relevantes.

Fonte: Databricks documentation: [What is Delta Lake?](#); [Delta table optimization](#).

7 · DESENVOLVIMENTO

Notebooks são ambiente de desenvolvimento

Notebooks do Databricks combinam código, texto e resultados. Eles suportam Python, SQL, Scala e R, colaboração e visualizações. Para estudos iniciais, Python e SQL cobrem a maior parte das atividades de engenharia de dados.

- Anexe o notebook a uma computação compatível antes de executar células.
- Use comandos mágicos, como %sql e %python, para alternar linguagens quando necessário.
- Separe lógica reutilizável em arquivos ou módulos em vez de concentrar tudo em um notebook.
- Versione código e mantenha parâmetros fora da lógica principal.

Fonte: Databricks documentation: Develop code in Databricks notebooks; Choose a development language.

8 · PRODUÇÃO

Produção exige orquestração e observabilidade

Um notebook executado manualmente ainda não é um pipeline de produção. Lakeflow Jobs organiza tarefas, dependências, agendas, parâmetros e tentativas de execução.

- Defina tarefas pequenas, com entradas e saídas claras.
- Configure dependências para impedir consumo antes da conclusão da etapa anterior.
- Use parâmetros para evitar cópias quase idênticas do mesmo código.
- Acompanhe falhas, duração, custo e qualidade das saídas.
- Trate CI/CD como promoção controlada de código e configuração entre ambientes.

Para cada falha, descubra primeiro se a causa está no código, nos dados, na configuração ou nos recursos de computação.

Fonte: Databricks exam guide 2026; Databricks documentation: Lakeflow Jobs.

9 · GOVERNANÇA

Governança começa pelo menor privilégio

Unity Catalog organiza ativos em uma hierarquia de metastore, catálogo, esquema e objeto. Permissões devem conceder somente o acesso necessário para cada identidade e tarefa.

- Tabelas gerenciadas têm ciclo de vida de dados e metadados controlado pela plataforma.
- Tabelas externas mantêm dados em uma localização externa definida.
- GRANT concede privilégios; REVOKE remove privilégios.
- Linhagem ajuda a compreender origens, transformações e consumidores.
- Credenciais e segredos não devem ser gravados diretamente em notebooks.

Governança não é apenas bloquear: ela permite que pessoas encontrem e usem dados confiáveis dentro de regras claras.

Fonte: Databricks documentation: Unity Catalog; Manage privileges.

Otimização começa pela evidência

Evite decorar uma única solução para qualquer lentidão. Observe métricas, plano de execução, volume de dados, distribuição entre partições e padrão de leitura.

- Muitos arquivos pequenos aumentam o custo de listagem e planejamento.
- Filtros seletivos reduzem a quantidade de dados lida.
- Junções podem exigir redistribuição de dados entre executores.
- Skew concentra trabalho em poucas partições e prolonga a etapa mais lenta.
- Logs e histórico de execução ajudam a separar sintomas de causas.

Regra de estudo: identifique o gargalo antes de escolher a otimização.

Fonte: Databricks documentation: Optimization recommendations; Spark UI.

Plano de estudo em quatro ciclos

Ciclo	Foco	Entrega prática
1	plataforma, lakehouse e Delta	criar workspace de estudo e uma tabela Delta
2	ingestão, SQL e PySpark	carregar CSV, limpar dados e comparar SQL/PySpark
3	medalhão, jobs e governança	pipeline bronze-prata-ouro com permissões
4	operação e revisão	investigar uma falha, revisar objetivos e responder simulados

Não avance apenas porque leu o tópico. Cada ciclo termina com algo executado, observado e explicado com suas próprias palavras.

Teste sua compreensão

- 1. Qual componente adiciona transações ACID às tabelas do lakehouse?
- 2. Em qual camada ficam os dados mais próximos da origem?
- 3. Qual é a diferença essencial entre batch e streaming?
- 4. Para que serve uma operação MERGE?
- 5. Por que funções nativas de Spark tendem a ser preferíveis a UDFs?
- 6. Qual serviço organiza tarefas e dependências de produção?
- 7. Qual componente centraliza governança de dados e IA?
- 8. O que schema enforcement evita?
- 9. Por que muitos arquivos pequenos podem prejudicar uma carga?
- 10. Qual deve ser o primeiro passo diante de uma consulta lenta?

Responda sem consultar as páginas anteriores. Depois explique por que as alternativas erradas estariam erradas; esse exercício prepara melhor para questões de cenário.

13 · REVISÃO

Gabarito comentado

- 1. Delta Lake.
- 2. Bronze.
- 3. Batch processa conjuntos delimitados; streaming trata eventos continuamente ou de forma incremental.
- 4. Combinar inserções, atualizações e, conforme a lógica, exclusões entre fonte e destino.
- 5. O otimizador compreende funções nativas e pode planejar sua execução.
- 6. Lakeflow Jobs.
- 7. Unity Catalog.
- 8. A gravação de dados incompatíveis com o esquema esperado.
- 9. Eles aumentam custos de listagem, abertura e planejamento.
- 10. Observar métricas e o plano para localizar o gargalo.

Use o gabarito para localizar lacunas, não para decorar frases.

Checklist antes da prova

- Rebaixei o guia oficial e confirmei que esta é a versão vigente para minha data.
- Consigo explicar lakehouse, Delta Lake e Unity Catalog sem consultar anotações.
- Pratiquei transformações equivalentes em SQL e PySpark.
- Construí ao menos um fluxo bronze-prata-ouro.
- Criei e acompanhei um job com mais de uma tarefa.
- Revisei permissões, tabelas gerenciadas e externas e linhagem.
- Consigo investigar uma falha usando logs e métricas.
- Treinei questões de cenário dentro do tempo disponível.

A certificação mede reconhecimento e aplicação de fundamentos. A prática reduz a dependência de memorização isolada.

FONTES

Referências essenciais

Guia oficial do exame - maio de 2026

<https://www.databricks.com/sites/default/files/2026-03/databricks-certified-data-engineer-associate-exam-guide-may-4-2026.pdf>

O que é um data lakehouse?

<https://docs.databricks.com/aws/en/lakehouse/>

Arquitetura medalhão

<https://docs.databricks.com/aws/en/lakehouse/medallion>

Delta Lake

<https://docs.databricks.com/aws/en/delta>

Notebooks do Databricks

<https://docs.databricks.com/aws/en/notebooks/notebooks-code>

Unity Catalog

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/>

Lakeflow Jobs

<https://docs.databricks.com/aws/en/jobs/>

Conteúdo produzido com apoio do Codex e revisado a partir de documentação oficial. A capa foi criada pelo gerador de imagens integrado ao Codex. Estrutura inicial inspirada no projeto educacional de Felipe Aguiar disponibilizado pela DIO.